

A Survey of Key Factors Affecting Software Maintainability

Balraj Kumar

Department of Computer Applications
Lovely Professional University, Punjab, India

E-mail: balraj_kr@yahoo.co.in

Abstract - Today software quality is a major point of concern and it is important to be able to assess the maintainability of a software system. Maintainability is the ability of the system to undergo modifications with a degree of ease. These changes could impact interfaces, components, and features when adding or modifying the functionality and meeting the new customer's requirements to cope with the changing environment. This paper describes the several types of vital factors that affect the maintainability of software systems as proposed by different researchers in different software maintainability models. These factors play a crucial role in the maintainability assessments. The maintainability models can be used to improve the quality of the software product so that maintenance can be performed efficiently and effectively.

Keywords: *Maintainability, factors affecting maintainability, maintainability model, maintenance, software quality.*

I. INTRODUCTION

One of the important concerns related to a software system is the cost of making a modification in it. During the maintenance of software systems, a large number of resources are expended. Thus, producing a software that is easily maintainable, may possibly save huge costs. The maintainability of a software system is affected by design and architectural principles [12]. If a software system is poorly designed for maintenance, the maintainability cannot be retrofitted to it later [13]. Whereas the increased maintainability can reduce the evolution and maintenance effort that in turn, enabling us to use the same resources to perform more changes, or accomplish the same change using fewer resources [14].

Both the terms maintainability and software maintenance are closely associated because maintainability refers to how easy it is to perform maintenance on the system. On the other hand, the process of changing software after it has been delivered and in use is called software maintenance [4]. *Software quality* can generally be described as the totality of characteristics of a software product

that could satisfy a given set of requirements. The various features include functionality, reliability, maintainability, portability etc. These characteristics are then broken down into lower level quality criteria or sub-characteristics which serve as attributes of software system.

The main objective of this paper is to explore the different categories of factors which influence the software maintainability and to review the study of various maintainability models. The factors affecting the maintainability are also known as the cost drivers as they are the indicators for deriving the cost to be expended during maintenance of the software systems [5]. The basic idea behind this paper is to present the researchers' perspectives on different factors and their impact on maintainability in the past years.

II. SOFTWARE MAINTAINABILITY

There are number of definitions of software maintainability available. Maintainability is one of the software quality aspects that reflects the effort required to perform the following:

- Elimination of latent errors
- Addition of new features/capabilities
- Removal of unused/undesirable features
- Revision of software

As per IEEE Standard Glossary of Software Engineering Terminology [1], maintainability can be defined as:

The ease with which a software system or component can be modified to correct faults, improve performance or other attributes, or adapt to a changed environment.

Maintainability (maintenance effort) is the most time consuming part of software life cycle which can range from 65% to 75% of total software development time [2]. Since majority of the time is spent during maintenance phase and it greatly affects the cost of software product. By putting more efforts for making the software maintainable during its development life cycle can significantly reduce the total software cost.

III. SOFTWARE MAINTENANCE

Software has to go through several changes throughout its lifetime. There are number of reasons causing these changes. They include:

- Changes in software requirements
- An erroneous software design
- Change in environment
- A change in hardware demands a change in the software too

Software maintenance is an important phase in the software life cycle. Its basic objective is to keep the software fully functional and up-to-date. Due to the changing requirements on performance over time, software products are modified to provide new functionalities or to change the existing ones. Software maintenance can be categorized into four types: adaptive, corrective, preventive and perfective maintenance [6]:

Adaptive maintenance is performed in order to adapt the software to a new environment, such as new hardware, new operating system, or a new database system. *Corrective maintenance* is performed to correct any faults in the analysis, design, implementation, documentation, or any other type of fault. A software system is said to be repairable or corrective if it allows the removal of residual errors present in the product while its delivery and the errors introduced in the software during its maintenance. *Preventive maintenance* is performed to keep the software product working and/or to extend its life. *Perfective maintenance* is performed to improve product efficiency by increasing its speed, adding/modifying the new/existing functionalities and enhancing the performance level.

Software maintenance team may or may not be the part of software development. To perform the maintenance tasks, they must understand how the software performs before they can modify it [3]. Therefore, the programs that are well documented, complete, up-to-date and accurate facilitate the learning process. The maintenance team can't maintain a program which is not understandable i.e. *the more difficult a program is to understand the more difficult it is to maintain and the more difficult it is to maintain, the higher its maintainability risk.*

IV. SOFTWARE QUALITY

Software engineering is a complex problem solving activity with quality goals [15]. There is a detailed software quality standard which is known as ISO 9126 [16]. This quality model presents three different views on software product quality:

Internal quality: Those properties of a system that primarily affects the software development organization and can be measured without executing it. Structural properties such as size or coupling are the examples of internal quality. *External quality:* Those properties of a system that are directly visible to a customer or client and can be observed during its execution. Maintainability and reliability are the examples of external quality. *Quality in use:* It concerns the properties experienced by its different types of users during operation and maintenance of the system. Internal quality is believed to impact external quality, which in turn impacts quality in use.

The ISO 9126 quality model defines a set of six main quality characteristics of software product quality and provides a framework for quality assessments. Fig.1 demonstrates these characteristics as well as the sub-characteristics:

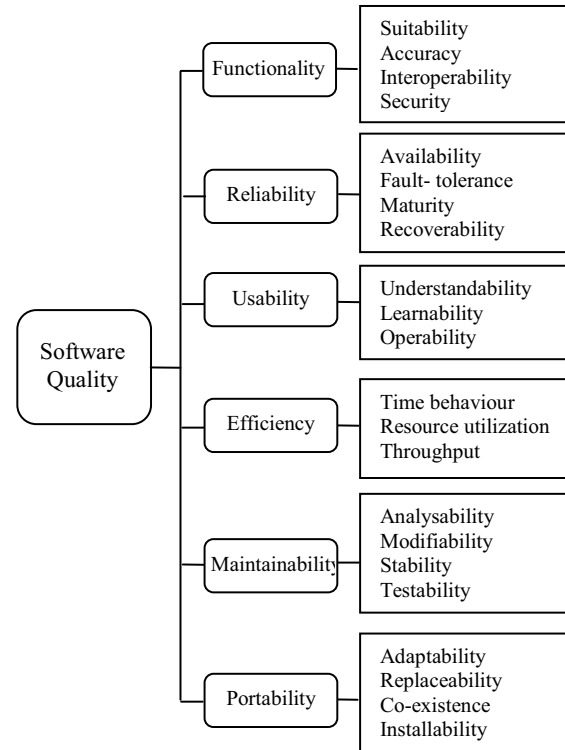


Fig.1: ISO 9126 Software Product Quality Characteristics and Sub-characteristics

V. MAINTAINABILITY MODELS

A large numbers of maintainability models have been proposed in the literature. Some models have assessed maintainability using the metrics from design as well as coding phase, while some are considering only design level metrics. These models include ISO 9126 software quality model, Boehm's quality model, McCall's model, Fuzzy model,

MEMOOD model etc. These models have applied different procedures and methodologies to predict the maintainability. For maintainability evaluations, different aspects of software system are taken into considerations which are termed as the characteristics of software because these are the factors that influence maintainability.

VI. FACTORS AFFECTING SOFTWARE MAINTAINABILITY

It is of great importance for a software client to be able to assess the maintainability of a software system. In literature, various researchers have presented and used several types of attributes to predict the maintainability. Here the paper intends to provide an insight into the various categories of factors which influence the software maintainability. So, let us take a brief overview of important contributory factors affecting the maintainability proposed by different researchers in the past:

- Maintainability is one of the six main quality characteristics as proposed by the ISO 9126 quality model [16]. The software maintainability is the software quality aspect that can best be approached at the software's design and code levels. Analysability, modifiability, stability and testability as described in Table-I are the important factors affecting the maintainability.

Specifically, it focuses on technical factors affecting the code, and to some extent on the documentation and tools that directly support development. To improve productivity and quality during maintenance, and to avoid moving to servicing prematurely, it is important to support all of these areas. A well-chosen selection of measures and guidelines for aggregating and rating them can provide a useful bridge between source code metrics and the ISO 9126 quality characteristics.

- David E. Peercy proposed a conceptual framework for software maintainability evaluation methodology that depends on six attributes [21]. These attributes are user-oriented general attributes of software which affect maintainability. They include modularity, descriptiveness, consistency, simplicity, expandability, and instrumentation as defined in Table-II.
- The researchers [17] presented a software maintainability model that can be used to improve the quality of the software product so that maintenance can be performed efficiently and effectively. This model designates a number of software related factors affecting maintainability of software systems, are explained in the Table-III.
- The researchers [18] proposed a four parameter integrated measure of software maintainability using a fuzzy model as discussed in Table-IV. This model integrates the four factors namely average number of *live variables*, average *life span* of variables, the *average cyclomatic complexity* and the *comments ratio* and provides a measure of maintainability.
- A large amount of experiential research has been done on measuring the structural properties of software. Some code or structural properties can also affect maintainability like size (lines of code) and complexity measures by McCabe [8] and Halstead [9]. Some researchers have tried to combine them into a single value, called maintainability index [10]. Anda has also used the structural measures (factors) to measure the maintainability [12] that have been listed in Table-V. Measures for inheritance, coupling and cohesion were suggested to deal with OO software [11].

TABLE-I: FACTORS REPORTED IN [16]

Factor	Description
Analysability	Analysability involves the effort needed to diagnose the deficiencies or causes of failure and to identify parts to be modified of the software system.
Modifiability	How much work needs to be done to implement a modification? The steps taken to make designs more adaptable and ease the implementation of changes will improve maintainability.
Stability	Stability is concerned with the risks associated with unexpected effects of modifications.
Testability	Testability refers to the ability to validate the modifications. It is a measure of how easy it is to create test criteria for the system and its components, and to execute these tests in order to determine if the criteria are met. Good testability makes it more likely that faults in a system can be isolated in a timely and effective manner.

TABLE-II: FACTORS REPORTED IN [21]

Factor	Description
Modularity	Modularity describes the logical partitioning of software into several parts, components, and modules. Software will be easy to understand and change when composed of independent modules.
Descriptiveness	The descriptiveness/documentation contains detailed explanations of the software design. It provides information regarding objectives, assumptions, inputs/outputs, processing, components, revision status etc.
Consistency	Consistency indicates that the software products associate and contain uniform notation, terminology and symbology. The use of naming conventions, I/O processing, error processing, coupling of modules etc are the reflections of consistency.
Simplicity	Software simplicity to the extent that it lacks complexity in organization, language, and implementation techniques and reflects the use of singularity concepts and basic structures.
Expandability	Expandability describes that a physical change to information, computational functions, data storage or execution time can-be easily accomplished. Parameterization of constants and basic data structure sizes generally improve expandability.
Instrumentation	Instrumentation characteristic contains aids which enhance software testing.

TABLE-III: FACTORS REPORTED IN [17]

Factor	Description
Readability	The degree to which a reader can quickly and easily understand source code.
Standardisation	A set of programming standards should be available to act as a guide in code writing to avoid idiosyncrasy among programmers.
Modularity	Decomposing a system into functional units, to impose hierarchical ordering on functional usage, to implement data abstractions, and to develop independently useful subsystems.
Programming language	A code written in a language where programmer not familiar with the results in difficulty in maintaining the reused code.
Level of validation and testing	More time and effort spent on design validation and program testing, results in fewer errors in program and consequently decreases maintenance cost resulting from error correction.
Complexity	The complexity of a software affects its maintainability. It is supposed to reflect to a certain extent, the difficulty in comprehending or maintaining codes.
Traceability	Traceability refers to the ability to trace a design representation or actual program components back to requirements.

TABLE-IV: FACTORS REPORTED IN [18]

Factor	Description
Average number of live variables	A live variable is said to be live at a specific statement if it is frequently referenced by the statements before or after that statement. Average of live variables is the sum of the total live variables divided by the total executable statements. This average in case of a program consisting of multiple modules can be expressed as the sum of averages of live variables of all modules divided by the total number of modules. Hence, <i>the more the average number of live variables, the more difficult it would be to develop and to maintain a software system.</i>
Average live variable span	The span is the number of statements between two successive references of a variable [19][20]. The average span size for a program consisting of multiple modules can be expressed as the sum of averages of span sizes of all modules divided by the total number of modules.
Comments ratio	Comment ratio can be described as the sum of total LOC and total number of comment lines. <i>The lower the comment ratio, the better is the readability, and the better the readability, the better is the maintainability.</i>
Average Cyclomatic Complexity	Average Cyclomatic Complexity can be expressed as the average of cyclomatic complexities of all modules.

TABLE-V: FACTORS REPORTED IN [12]

Factor	Description
Design for the Problem Domain	The complexity of the problem domain must justify the choice for the design. The use of design patterns may make the maintenance process more easy and simple. A good design may prove to be a good solution only if the developers are competent enough.
Appropriate Comments	Comments should be expressive and meaningful, so that they could enhance the readability of the code.
Naming Conventions	The usage of standard naming conventions for packages, classes, methods and variables makes the understanding easier.
Objects and classes	The software system requirements generally designate several primary objects that are implemented in the form of classes. The mapping of these classes from requirements to coding should be simple.
Use of Components	Classes should be organized according to functionality or according to the layer of the code on which they operate.
Technical Platform	This is concerned with poor or no documentation, inherent requirements that evolve when a system switches from one environment to the other. The non-standard third party components may pose a challenge to using and replacing the components in further development.
Simplicity	Software size and its complexity is a very critical issue. The code smell "DuplicateCode" is, perhaps, the worst [7]. It takes longer to identify a specific class when there are many classes. The presence of several classes that are almost empty is a sign of code that may possess low maintainability.
Encapsulation	Since Java methods introduce dependencies, such as creating object structures before a method is invoked, this can lead to maintenance problems.
Inheritance	Maintainability may get affected the way the inheritance is used. Thus, the inheritance should be used carefully. The multiple inheritances may involve several classes, which may lead to confusion and low maintainability.
Proprietary Libraries	The use of proprietary libraries and coding style of the developers of these libraries may result in a lower maintainability, because the staff responsible for maintaining the system will need to acquaint themselves with them.
Layered Architecture	A clear distinction among presentation, business and persistence layer is considered as a good practice. Each layer should be independent from the layers above it and depend only on more general components in the lower layers.

Besides the properties of code, maintainability may also be affected by a several number of other factors, like skills of the staff performing the maintenance, the maintenance tasks to be performed on the system, the tools used, advancements in technology means new standards for design and architecture of software systems, the application being supported, staff mobility, too many versions etc. The maintenance tasks may vary. Small change tasks restricted to a limited part of the system may be easier to perform on software with fewer and larger classes, because it becomes easy to identify the class or cluster of classes where changes are to be made. On the other side, larger change tasks that require changes to larger parts of the software system depend more on the organization of the code and the programming principles used.

VII. CONCLUSION

Software maintainability is the ease with which a software system or a component can be modified. In software engineering, maintainability is required for correction of defects, meeting new requirements, to cope with a changing environment, and modifying a software product with ease. Maintainability can also

affect the time it takes to restore the system to its operational status following a failure or removal from operation for an upgrade. So we can say, in order to identify and fix a fault within software, maintainability is required.

In the coming days, more importance needs to be given to the contributory factors which affect the maintainability. Proper analysis and use of these factors may make the maintainability assessment easier and clearer. If the maintainability aspect is well taken into consideration while developing the software in its early stages, the software would be highly maintainable and thus, this would reduce the maintenance efforts greatly.

REFERENCES

- [1] IEEE, IEEE Standard Glossary of Software Engineering Terminology, report IEEE Std 610.12-1990, IEEE, 1990.
- [2] S. Muthanna, K. Konotogiannis, K. Ponnambalam, and B. Stacey. A maintainability model for industrial software systems using design level metrics. In Seventh Working Conference on Reverse Engineering, pages 248–256, November 2000.
- [3] G. M. Berns, "Assessing software maintainability," Communications of the ACM, vol. 27, no. 1, pp. 14–23, Jan. 1984.

- [4] I. Sommerville (1996): Software Engineering, 5th Ed. Addison Wesley, USA , 1996
- [5] W. Hordijk and R. Wieringa, "Surveying the Factors that Influence Maintainability", *ESECFSE' 05*, September 5-9, 2005, Lisbon, Portugal.
- [6] S. O. Olatunji, Z. Rasheed, K. A. Sattar, A. M. Al- Mana, M. Alshayeb and E. A.El-Sebakhy, "Extreme Learning Machine as Maintainability Prediction Model for Object-Oriented Software Systems", *Journal of Computing*, vol. 2, no. 8, pp. 49-56, 2010.
- [7] Fowler, M. and Beck, K, "Bad smells in code. In: *Refactoring: Improving the design of existing code*", 1st ed. Addison-Wesley, Boston, pp.75-88, 2000.
- [8] T. McCabe, "A Complexity Measure," *IEEE Transactions Software Engineering*, vol. SE-2, no. 4, pp. 308-320, 1976.
- [9] M. H. Halstead, *Elements of Software Science* (Operating and programming systems series). Elsevier, 1977.
- [10] K. D. Welker, "Software Maintainability Index Revisited," *CrossTalk - J. of Defense Software Engineering*, 2001.
- [11] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, 1994.
- [12] B. Anda, "Assessing Software System Maintainability using Structural Measures and Expert Assessments," in *IEEE International Conference Software Maintenance*, 2007, pp. 204-213.
- [13] Kosciński A, Candido Bracarense Costa J, Combining Analytical Hierarchical Analysis with ISO/IEC 9126 for a Complete Quality Evaluation Framework, *International Symposium and Forum on Software Engineering Standards*, 1999, pp. 218-226.
- [14] Swanson B E, *IS Maintainability: Should It Reduce the Maintenance Effort?* SIGCPR 1999, New Orleans LA, USA.
- [15] Boehm,B et al. In (1996) "Identifying Quality-Requirement Conflicts," *IEEE Software*, vol. 13, pp. 25-35.
- [16] ISO/IEC 9126 International Standard ISO/IEC 9126: Software Engineering - Product Quality.
- [17] *Khairuddin Hashim and Elizabeth Key*, "A Software Maintainability Attributes Model", *Malaysian Journal of Computer Science*, Vol. 9 No. 2, December 1996, pp. 92-97
- [18] K.K. Aggarwal, Yogesh Singh, Pravin Chandra and Manimala Puri, "Measurement of Software Maintainability Using a Fuzzy Model", *Journal of Computer Sciences* 1 (4): 538-542, 2005 ISSN 1549-3636 © 2005 Science Publications
- [19] Elshoff J.L.L., 1978. An investigation into the effects of the counting method used on software science measurements. *ACM SIGPLA Notices*, 13: 30-45.
- [20] K.K Aggarwal. and Yogesh Singh, "A modified approach for software science measures", *ACM SIGSOFT Software engineering Notes*, 1994, USA.
- [21] David E. Peercy, "A Software Maintainability Evaluation Methodology", *IEEE Transactions on Software Engineering*, Vol. SE-7, NO. 4, July 1981.